INTERNAL TOOL GUIDE · QE, DEV & PRODUCT

Onsite Validation Hub

A single-file browser tool for testing Medallia/Kampyle onsite feedback forms before they ship — targeting, design, accessibility, performance, privacy and security, all verified live against a real property, never mocked.

0 external dependencies | **22** diagnostic panels | **5** workspaces

<2s to connect a property | Runs from **one HTML file**

3½-minute walkthrough — silent, captioned, recorded against a live property. Includes a real form submission end to end. No narration, just the real tool.

► The walkthrough video is not playable in a PDF — open docs/onsite-hub-walkthrough.mp4 in the repository, or the web version of this guide.

WHY THIS EXISTS

What this tool actually is

Testing an onsite feedback form usually means guessing why it isn't showing, editing cookies by hand, and hoping the accessibility and performance are fine. This tool answers those questions directly, against the real SDK, in one tab.

You paste the property's embed snippet into the box on the dashboard and click **Inject Property**. From that moment, everything downstream — which forms are eligible to show, whether their design matches configuration, whether they're accessible, how they perform, and whether they leak anything sensitive — is computed from what the real SDK is doing on this page, not from a static config dump. If a check can't be verified confidently (usually because content renders inside a cross-origin iframe this page can't read into), it says so plainly instead of guessing.

For QE and Dev: this replaces manually flipping localStorage keys, wondering why targeting rules don't match observed behaviour, and eyeballing contrast ratios. **For Product:** the status bar and one-click bug report give a plain-English read on whether a property is healthy without needing to understand the SDK at all.

---

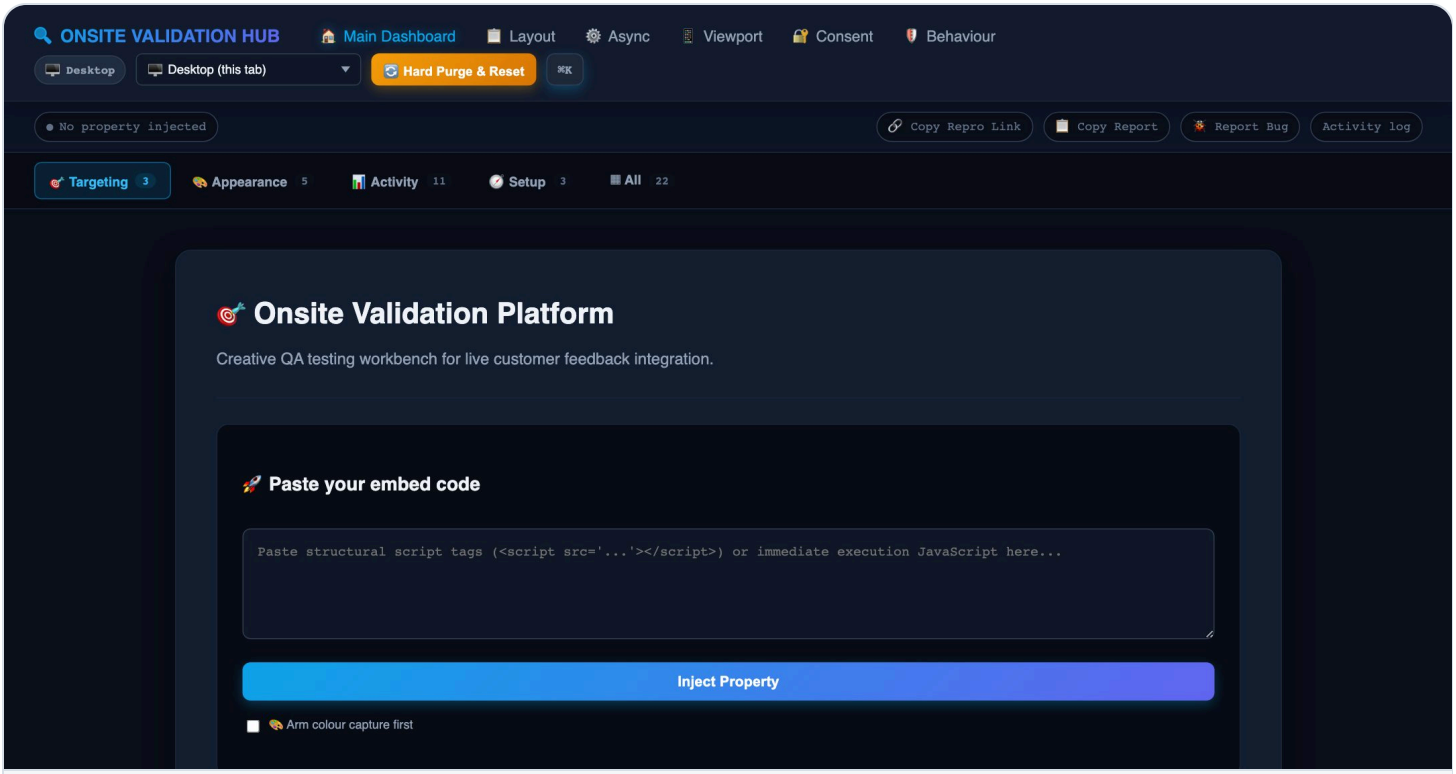
3 STEPS, UNDER A MINUTE

Quick start

You need the property's embed snippet — the same `<script>` tag that would go on the real site.

1. Open the dashboard

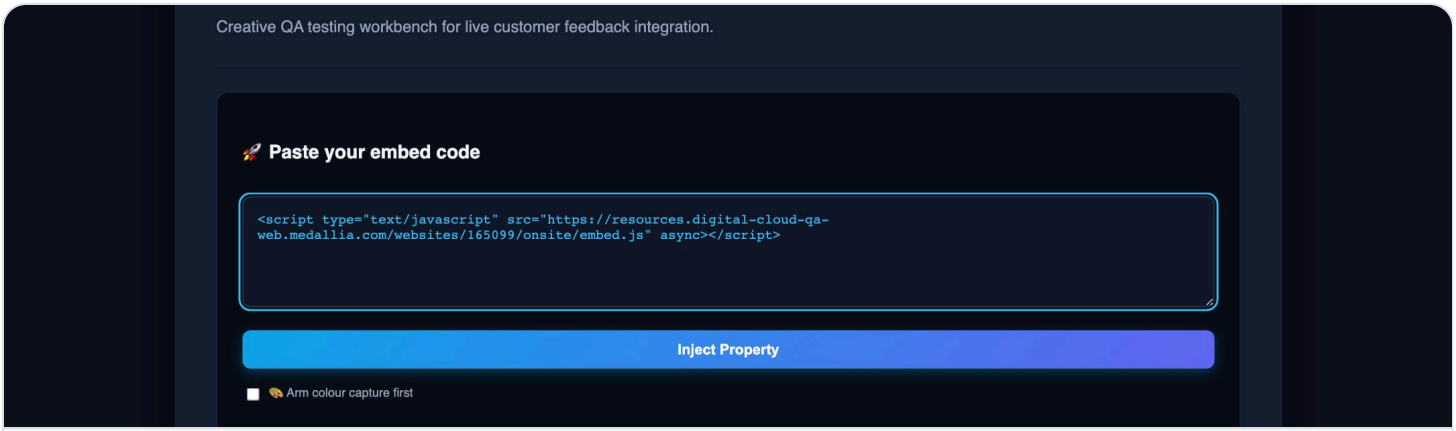
It starts empty — nothing is injected, nothing is being tracked yet.



The dashboard before anything is injected.

2. Paste the embed snippet

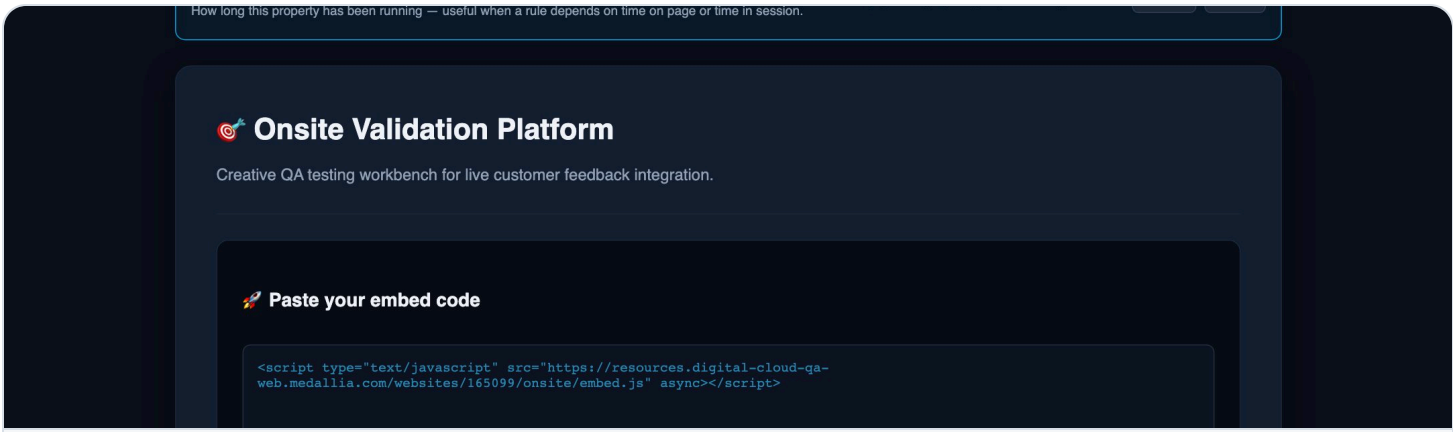
Drop the full `<script>` tag into the box. It accepts a structural tag or immediate-execution JavaScript.



A real embed snippet, ready to inject.

3. Click Inject Property

The SDK typically connects in under two seconds. The message under the button only turns green once the SDK has actually confirmed itself — a script that 404s or points at an unpublished property is reported as a failure, not a false success.



Confirmed connection — property ID and connect time shown.

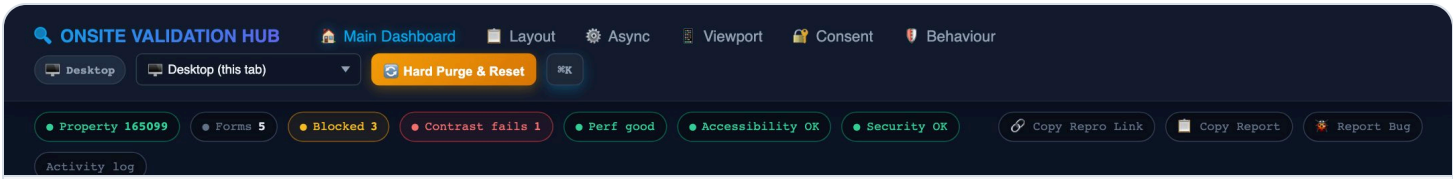
TIP

Everything the tool observes is scoped to the time since this injection. Use **Hard Purge & Reset** in the top-right whenever you want a completely clean slate — new visitor, no history, no stored state.

THE ONE-GLANCE SUMMARY

Reading the status bar

Once a property is injected, the bar under the navigation summarises everything below it. This is the fastest way to know if a property is healthy without opening a single panel.



Property ID, form count, and a live summary of every audit — updated continuously.

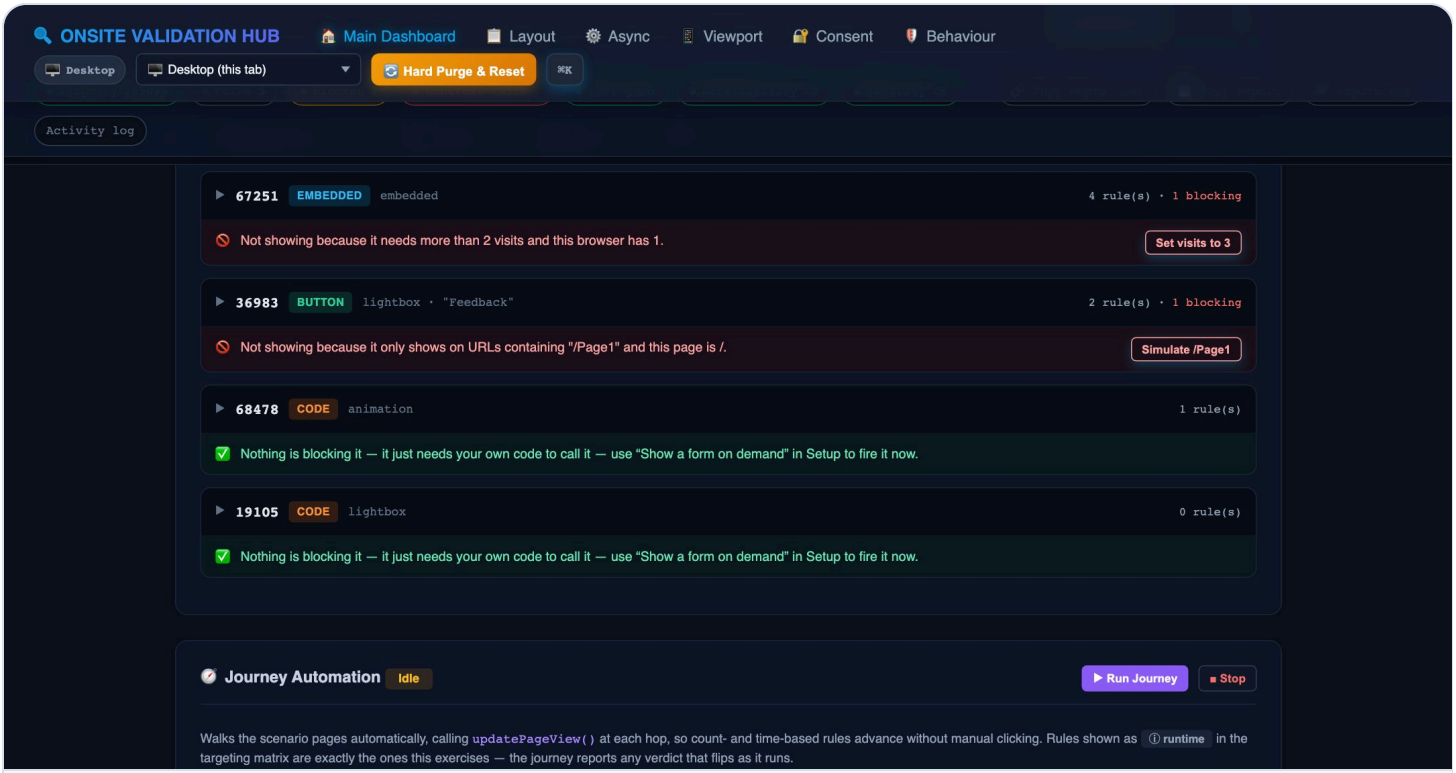
Each chip is clickable and jumps straight to the panel it summarises, switching workspace if needed. A chip turns amber or red the moment something needs attention; a clean grey/green row means nothing does. To the right, **Copy Repro Link**, **Copy Report** and **Report Bug** turn the current state into something you can hand to someone else — covered under For PMs & stakeholders.

THE FEATURE TO LEARN FIRST

The Targeting Matrix

`Panel: Will each form show?` TARGETING

This answers the single question everyone actually asks: **will this form show right now, and if not, why not?** Every published form gets a plain-English verdict, evaluated live against the real stored state — not a static reading of config.

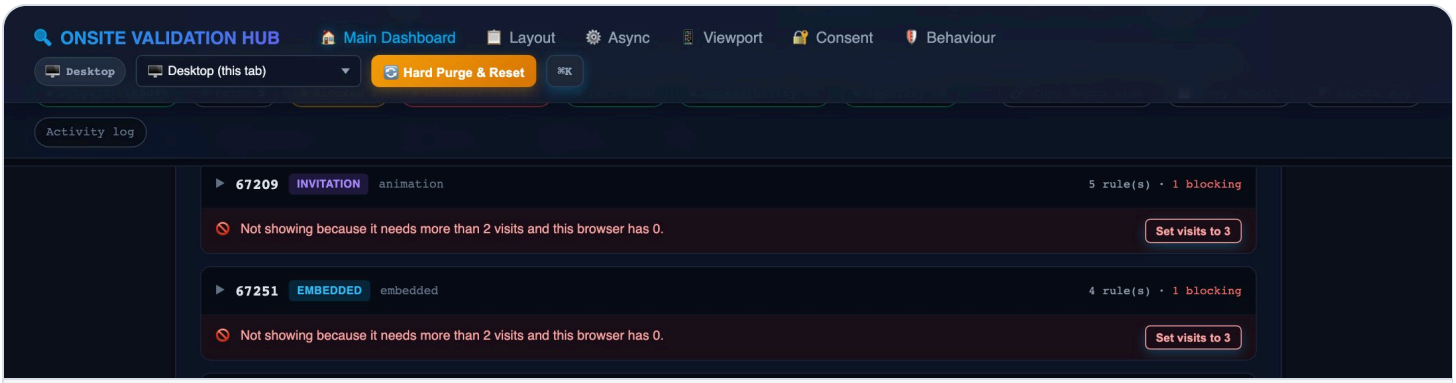


Every form on the property, each with its own live verdict.

- Eligible — nothing is blocking it
- Waiting on something only real behaviour can trigger
- Blocked — with the exact reason

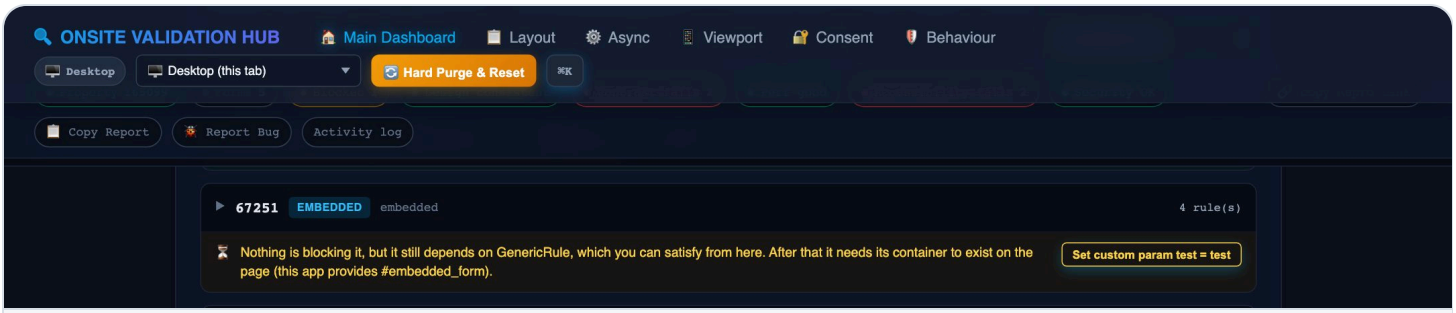
Walking through a real block

Here, form `67251` requires **more than 2 visits**, and this browser has 1. The matrix doesn't just say "blocked" — it names the exact rule, the exact stored value, and offers a button that sets precisely the value the rule needs.



Blocked, with the reason and a one-click fix — not a guess, the exact value the rule needs.

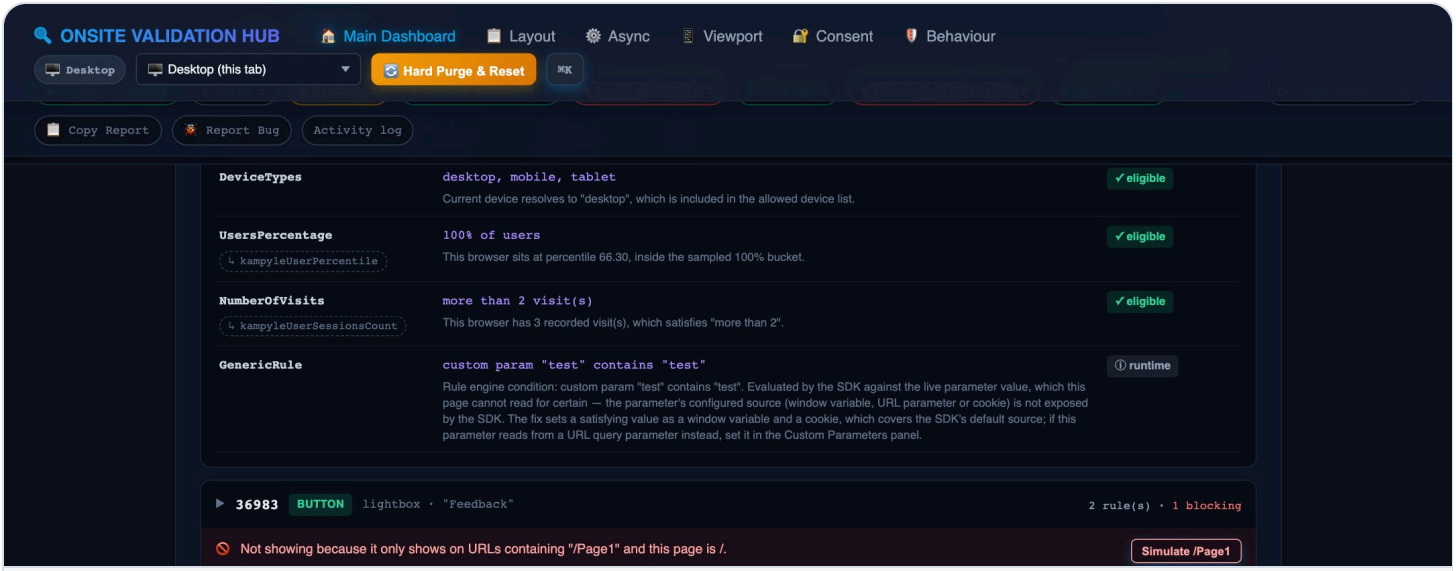
Click it, and the SDK is asked to re-evaluate targeting immediately. The visit rule clears — and because this form also has other dependencies, the matrix reveals the next real one underneath it rather than declaring victory early.



One dependency down — the next one (a rule-engine condition) surfaces automatically.

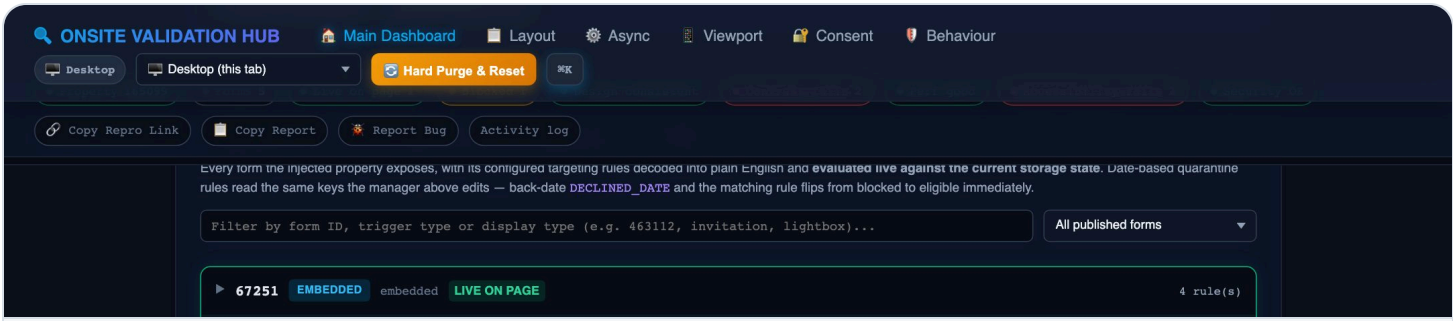
Rule-engine conditions, in English

Rule-builder conditions — arbitrary combinations of custom parameters, operators and values — used to render as raw JSON with no way to act on them. Expand any form card and they read as a sentence instead, and where a satisfying value can be derived, a fix button sets it.



“custom param "test" contains "test"” — read from the rule builder's own output, not guessed.

One more click, and the form is confirmed **live on the page** — not because a verdict flipped to green, but because the SDK itself reports the form as shown.



Confirmed by the SDK: **LIVE ON PAGE**, not just "nothing blocking."

WHAT THE FIXES ACTUALLY TOUCH

Every one-click fix writes to the same storage keys (or window/cookie values, for rule-engine conditions) the real SDK reads — then calls `updatePageView()` so targeting is re-evaluated immediately, the same way it would for a returning visitor. Nothing is faked or short-circuited inside the SDK.

ONE HONEST LIMIT

For a rule-engine condition, the SDK reads a parameter's value from one of three sources — a window variable, a URL parameter, or a cookie — and which one a given parameter uses isn't exposed anywhere the page can read. The fix writes the window variable and the cookie, which covers the default and the common case. If a parameter is configured to read from the URL instead, set it from the **Custom Parameters** panel in Setup.

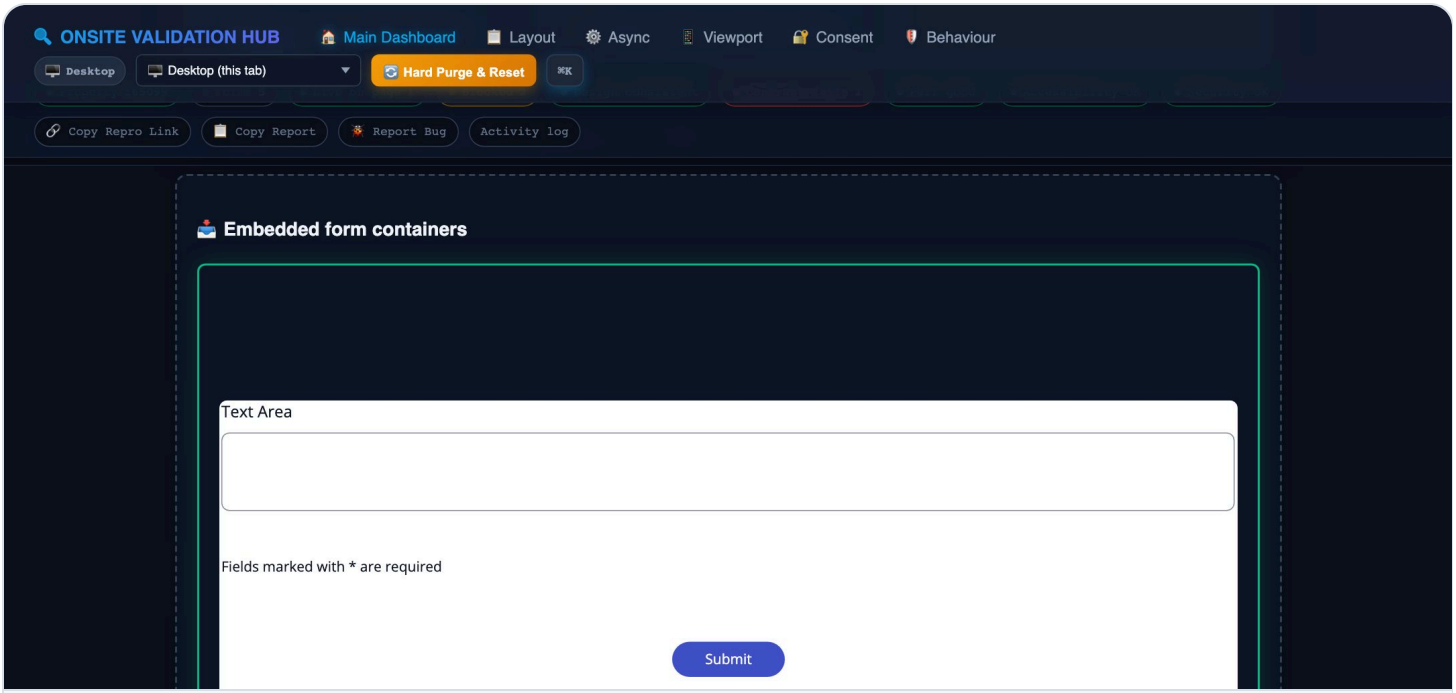
FROM VERDICT TO REAL EVIDENCE

Seeing the form — and submitting real feedback

A green verdict is only half the story. The point of clearing targeting is that the form actually appears, a respondent can complete it, and you can prove the response landed. All three are visible in this tool.

1. The form genuinely renders

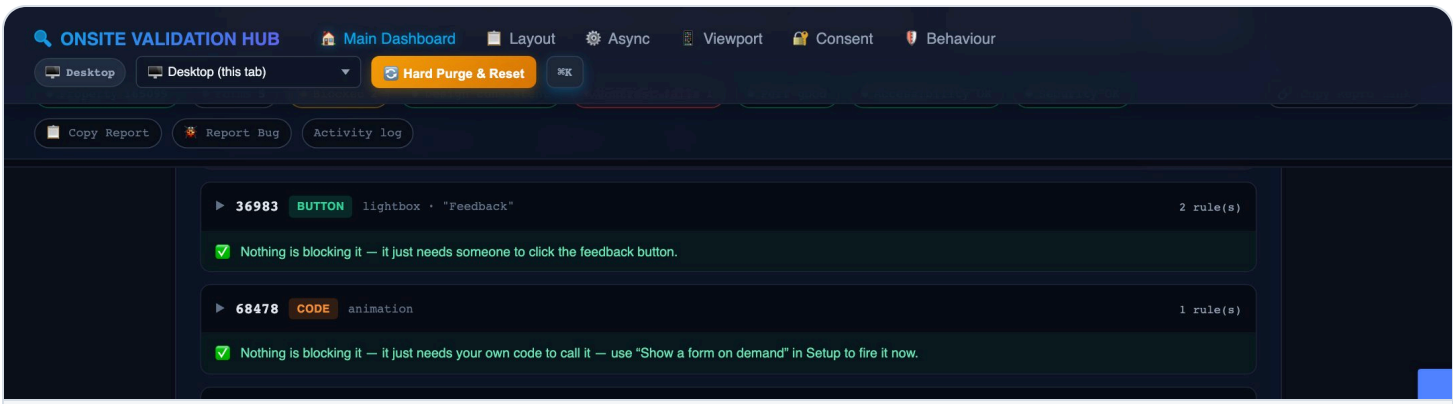
Once the embedded form's rules are satisfied, it renders into the page's container for real — this is the live Medallia form, not a preview or a mock-up.



Targeting satisfied means the form appears — rendered into `#embedded_form` on the page.

2. URL-scoped forms behave the same way

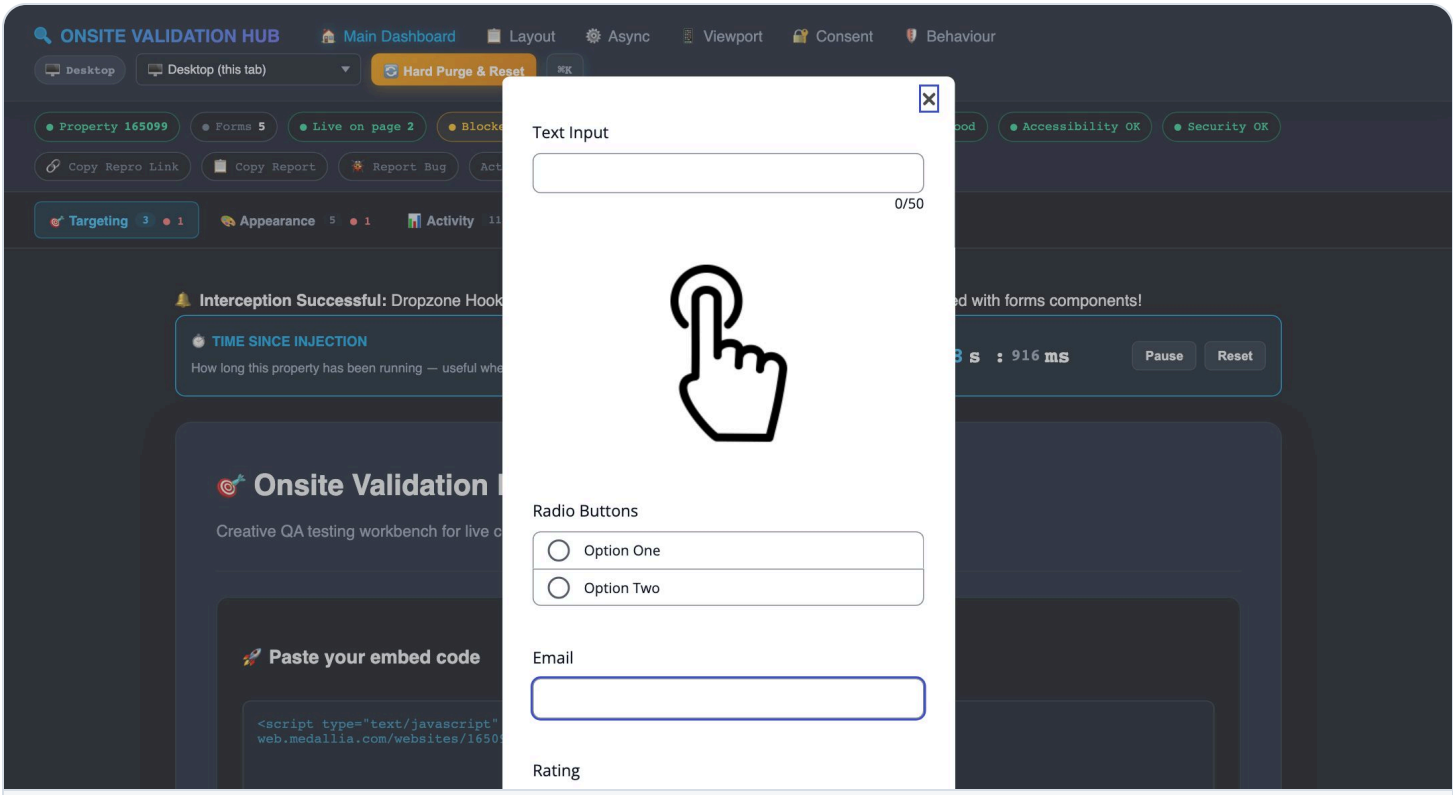
The feedback-button form only shows on `/Page1`. Simulate that URL and its rule clears — the matrix drops to “it just needs someone to click the feedback button,” and the real button appears pinned to the right edge of the page.



Rule cleared. The remaining step is a genuine user action, and the matrix says so.

3. Open and complete it as a respondent would

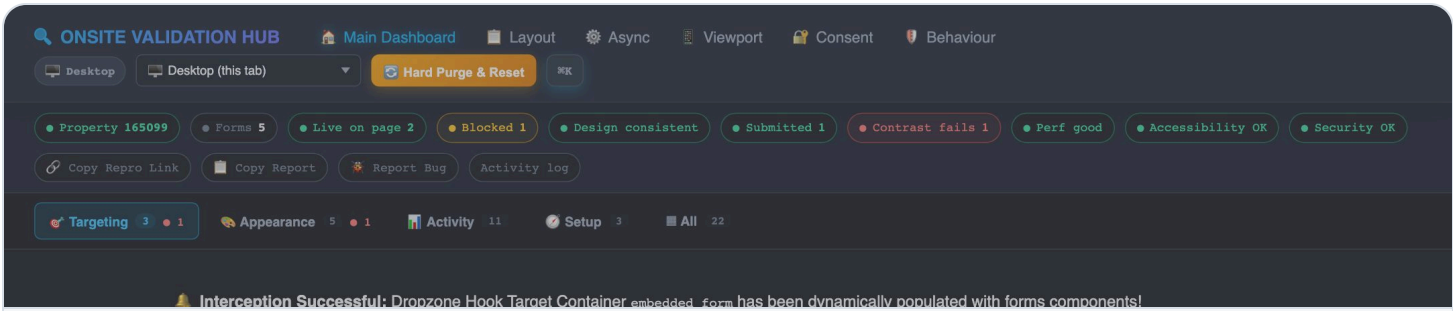
Clicking the button opens the real form. You can fill it in and submit exactly as a customer would — nothing about this path is simulated.



The genuine feedback form, opened by clicking the real button.

4. A “Submitted” chip appears the moment it lands

As soon as the SDK reports the submission, a **Submitted** chip appears in the status bar. It's a live count, and it's clickable.

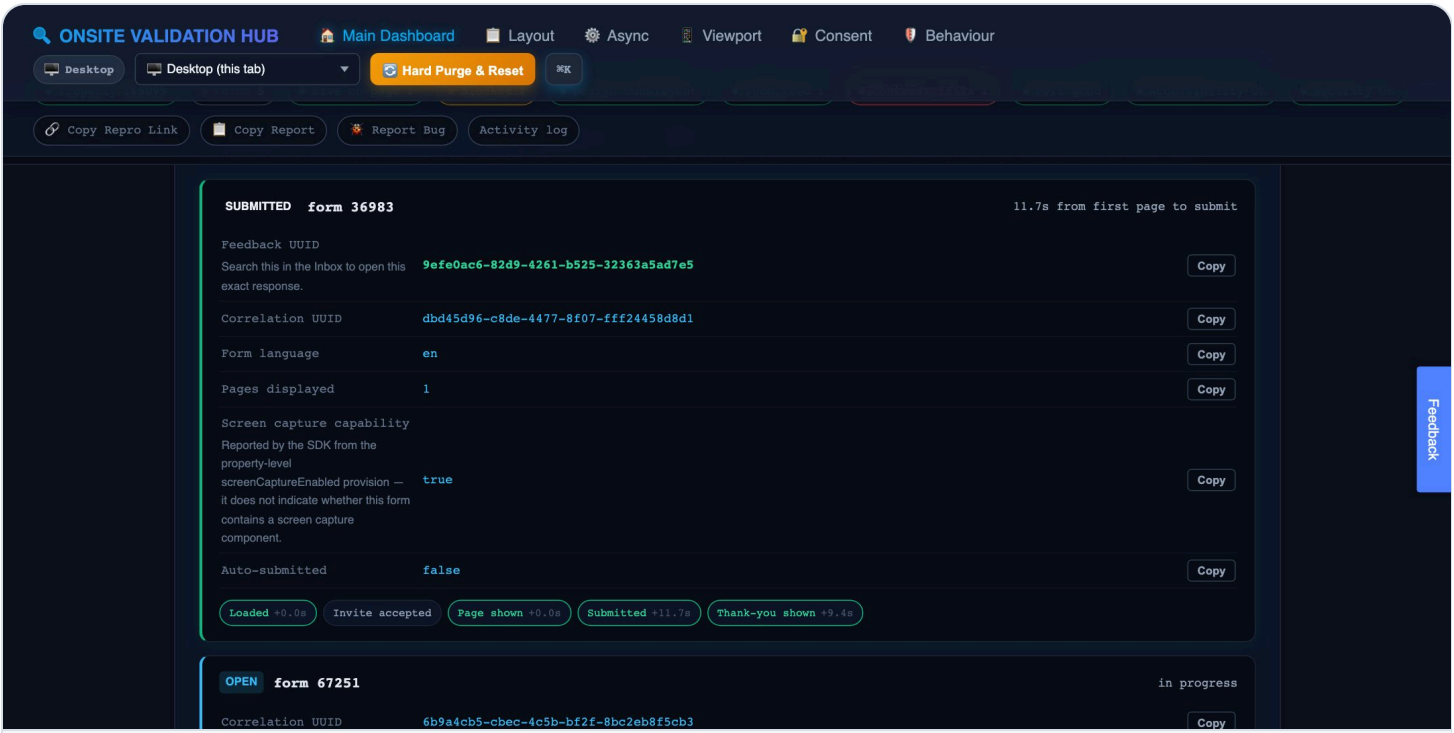


Confirmation at a glance — no need to go looking for it.

5. Click it to see exactly what was captured

`Panel: Feedback Submission Inspector` ACTIVITY

The chip jumps straight to the Submission Inspector, which reconstructs the whole journey from the SDK's own events — every step the respondent took, with real timings.



The full record of a single submission, captured live.

CAPTURED	WHAT IT'S FOR
Feedback UUID	Search it in the Medallia Inbox to open this exact response. This is the link between a test you ran and a real record.
Correlation UUID	Ties the browser session to the SDK's own telemetry for this journey.
Form language	The language the form itself reported on submit — checked against the page's `lang` in the accessibility audit.
Pages displayed	How many form pages the respondent actually saw before submitting.
Screen capture capability	The property-level `screenCaptureEnabled` provision as reported by the SDK.
Auto-submitted	Whether the submission was triggered programmatically rather than by a person.
Step timeline	Loaded → invite accepted → page shown → submitted → thank-you shown, each with its offset from the start.

WHY THE FEEDBACKUUID MATTERS MOST

It's the one value that turns “I tested it” into “here is the record.” Paste it into the Medallia Inbox and you land on the exact response your test produced — which is what makes a QA result verifiable by someone who wasn't watching your screen.

WHAT ISN'T CAPTURED

The respondent's actual typed answers are **not** visible here. The form renders in a cross-origin iframe, so this tool only sees what the SDK reports over the network — the journey and its metadata, never the content of someone's responses. That's also why the PII scan can only inspect outbound payloads rather than what a person typed into the form.

FINDING YOUR WAY AROUND

Five workspaces, grouped by what you're trying to find out

22 panels is a lot to scroll through, so they're grouped by the question you're actually asking — not by feature category. Switch with the tab bar, or jump straight there with ⌘ K.

🎯

Targeting

Will each form show, and why not. The matrix, lifecycle state, journey automation.

🎨

Appearance

Does it look and behave right — design, contrast, accessibility, device preview.

📊

Activity

What's happening live — events, submissions, performance, security, bug reports.

🛠️

Setup

Custom parameters, manual SDK commands, viewport controls, environment comparisons.

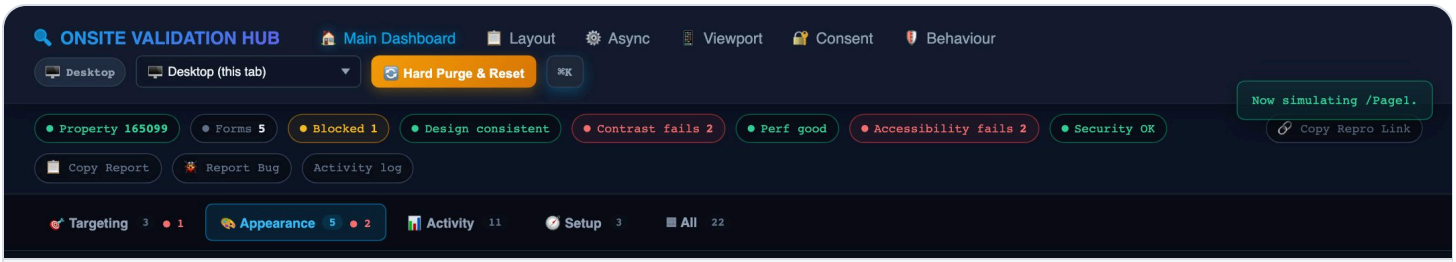
▦

All

Every panel on one screen — useful for a full pass before a release.

WORKSPACE

Appearance — does it look and behave right

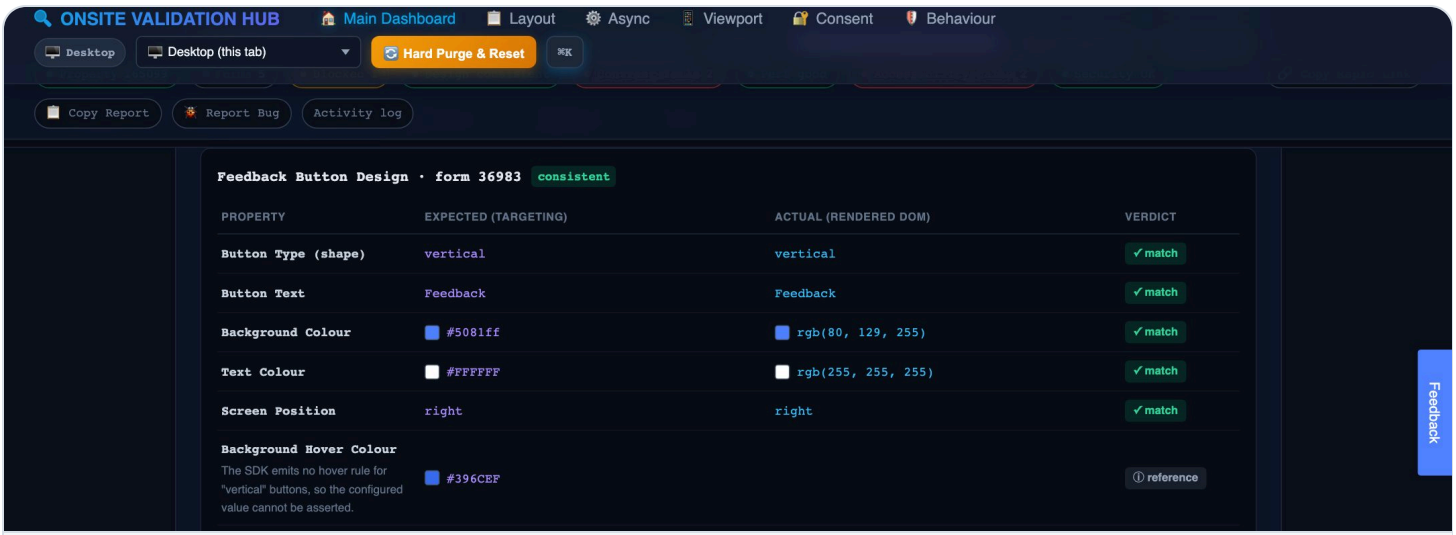


Selected from the workspace bar.

Design & Display Render Validator

`Panel: Design & Display Render Validator`

Compares what a form is *configured* to look like against what actually rendered — button shape, colours, display mode (popup vs. lightbox vs. embedded). Catches the class of bug where config says one thing and the live property shows another.

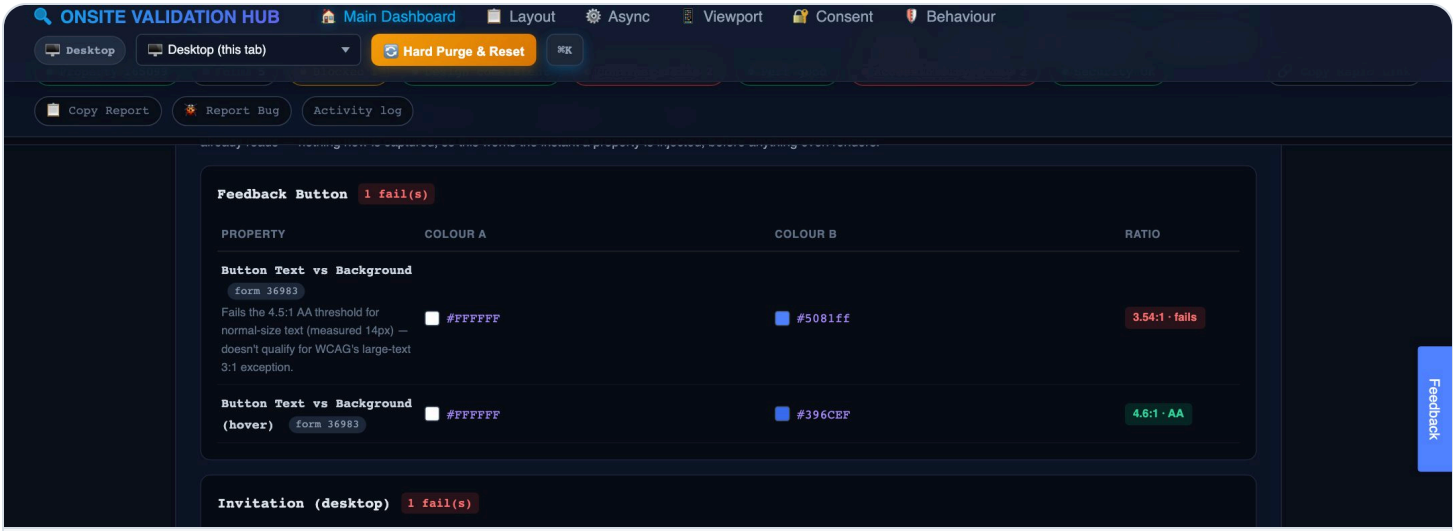


Expected (from targeting) vs. actual (from the rendered DOM), side by side.

WCAG Contrast Audit

`Panel: WCAG Contrast Audit`

Measures contrast from **real rendered pixel colours**, not from configuration — which is the only way to catch a mismatch between what was configured and what a browser is actually painting. Every ratio is checked against the WCAG AA threshold for its text size.

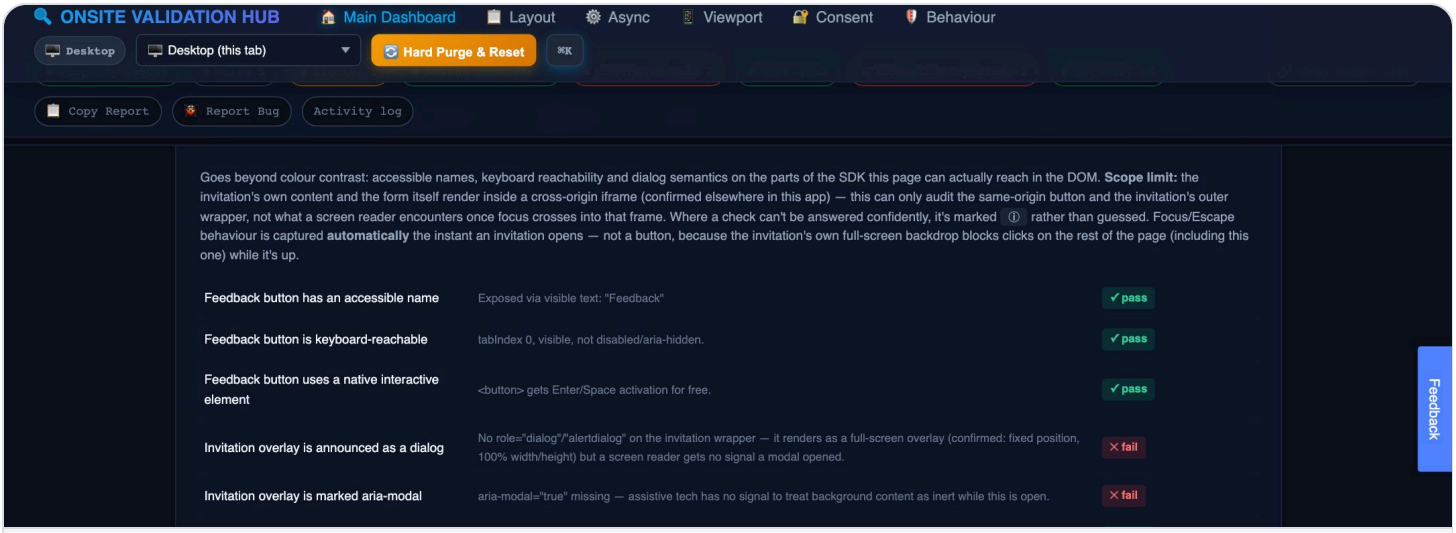


Real measured ratios — a config-only check would miss a rendering-level mismatch entirely.

Full Accessibility Audit

`Panel: Full Accessibility Audit`

Goes beyond colour: accessible names, keyboard reachability, native interactive elements, and dialog semantics on the invitation overlay. Every check is verified against the live DOM; where something can't be judged confidently (usually because it's behind a cross-origin iframe boundary), it says so instead of guessing at a pass or fail.

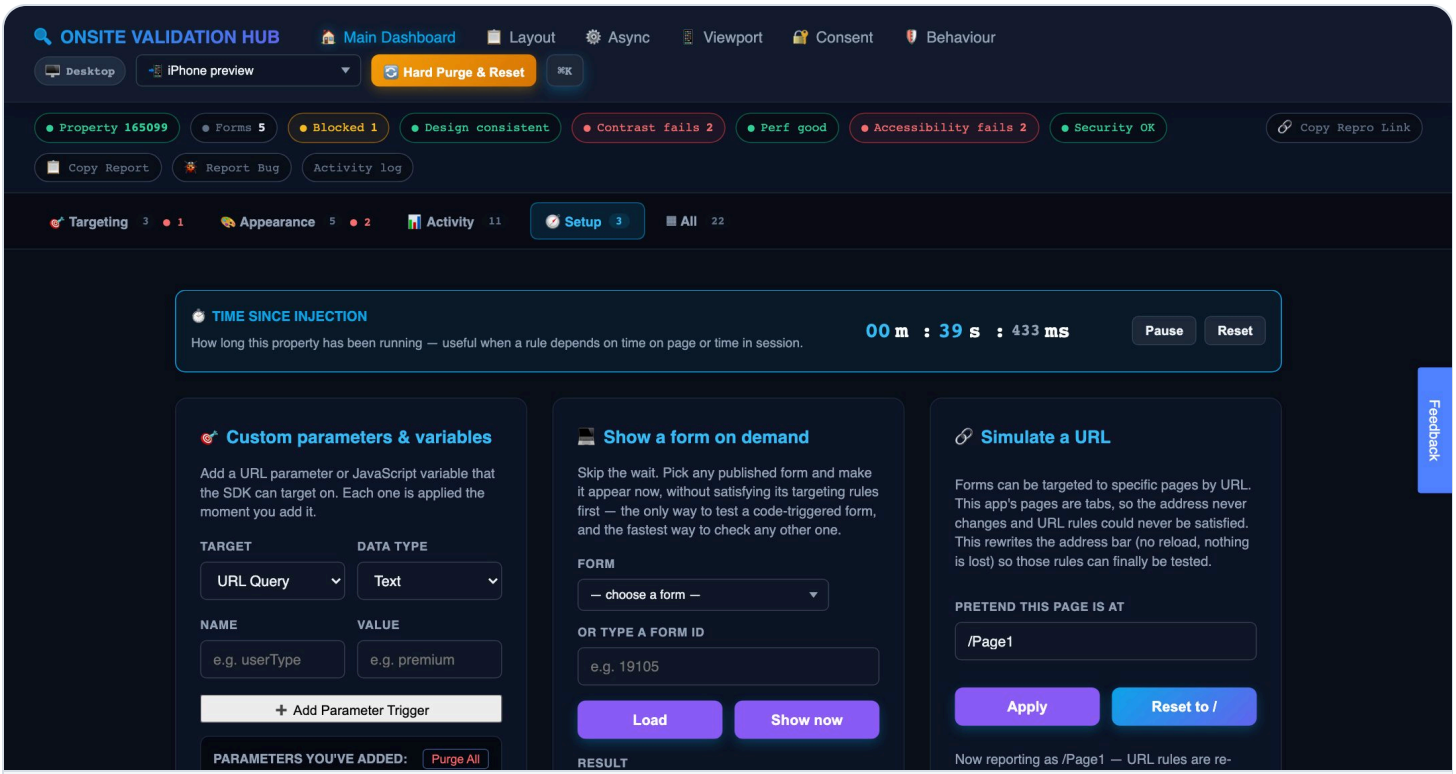


Keyboard reachability, ARIA roles and accessible names — all checked live.

Live Device Preview

`Panel: Live Device Preview`

Renders the whole sandbox inside a real device-sized viewport, so `position:fixed` anchors, responsive breakpoints and mobile invitation layout resolve exactly as they would on the actual handset — not approximated.



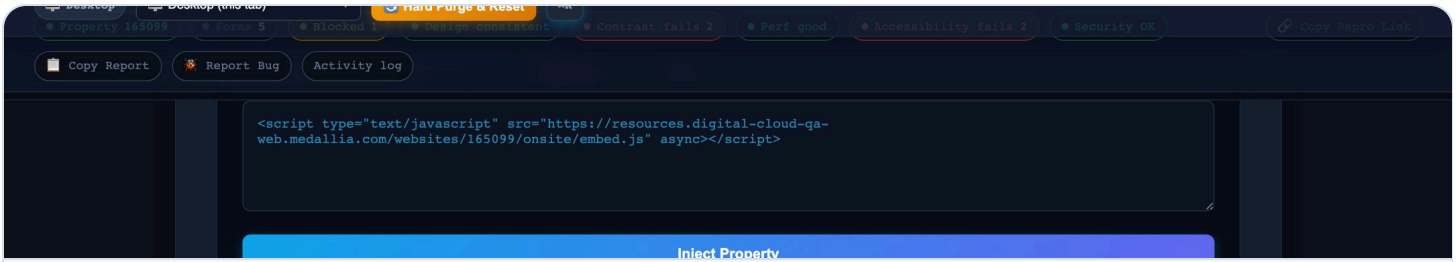
A genuine iPhone-sized viewport, with the real invitation rendering inside it.

PREVIEW VS. REAL MOBILE

This checks *layout* — it does not change what the SDK thinks the device is for targeting or submission attribution. To test whether a form actually shows on mobile or a submission is correctly attributed to a phone, use the device dropdown's **Launch Chrome** options, which drive a real emulated Chrome session via DevTools Protocol. See Known limits for exactly why the two are different.

WORKSPACE

Activity — what's happening live



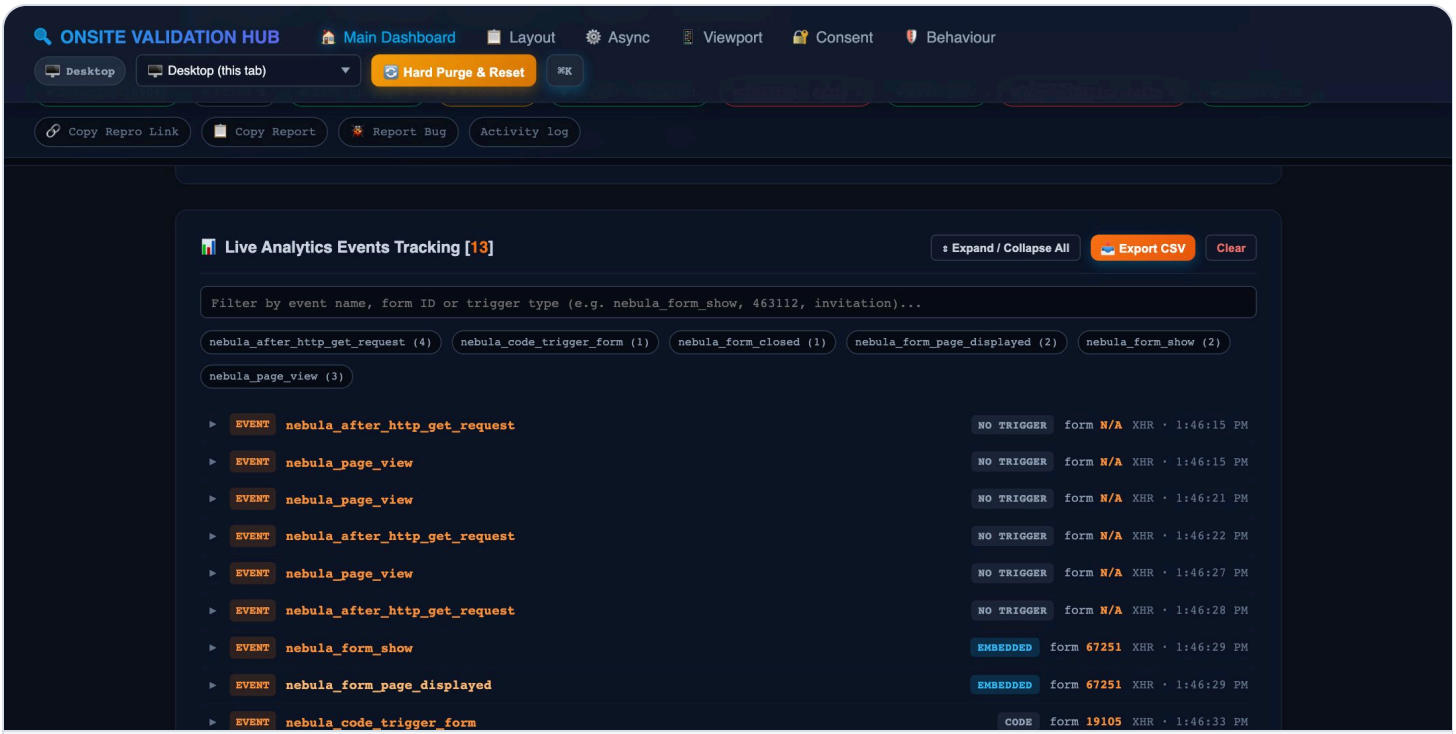
Selected from the workspace bar.

This is the busiest workspace — live event streams (analytics, the SDK's own custom event bus, a unified timeline of everything), plus the audits that make sense to check periodically rather than once: performance, privacy, security, and diffing findings against a prior run.

Live Analytics Events Tracking

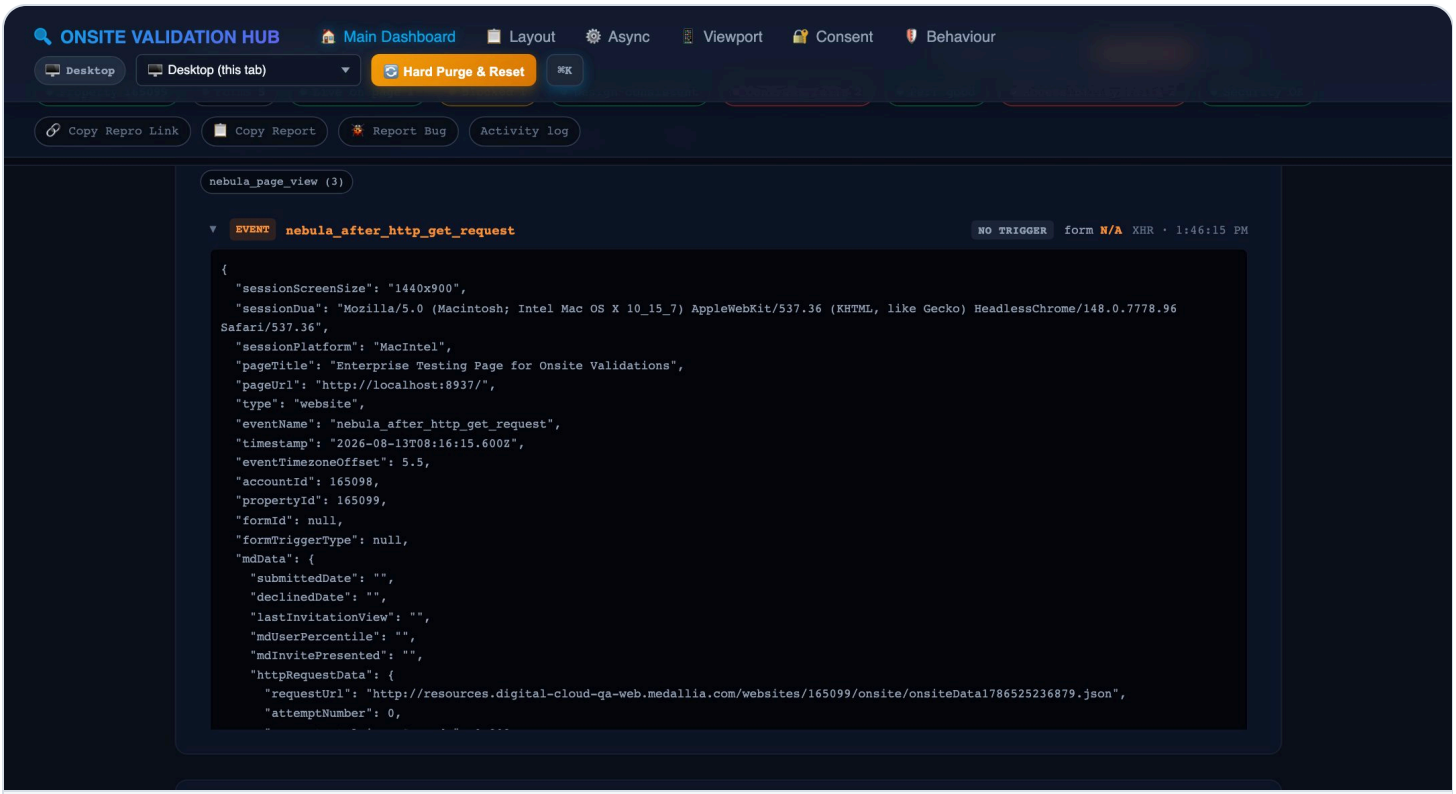
`Panel: Live Analytics Events Tracking`

Every analytics payload the SDK sends, captured as it goes out. The chips above the stream group events by name with a live count, so you can see at a glance what the property is actually emitting — and spot anything firing far more often than it should.



Each event carries its trigger type and form ID, so you can tell which form produced it.

Click any row to expand the full JSON payload — the whole thing, exactly as sent. This is where you confirm a custom parameter actually reached Medallia, or check what the SDK reported about the session and device.



The complete payload, scrollable in place. **Export CSV** takes the whole stream out for analysis.

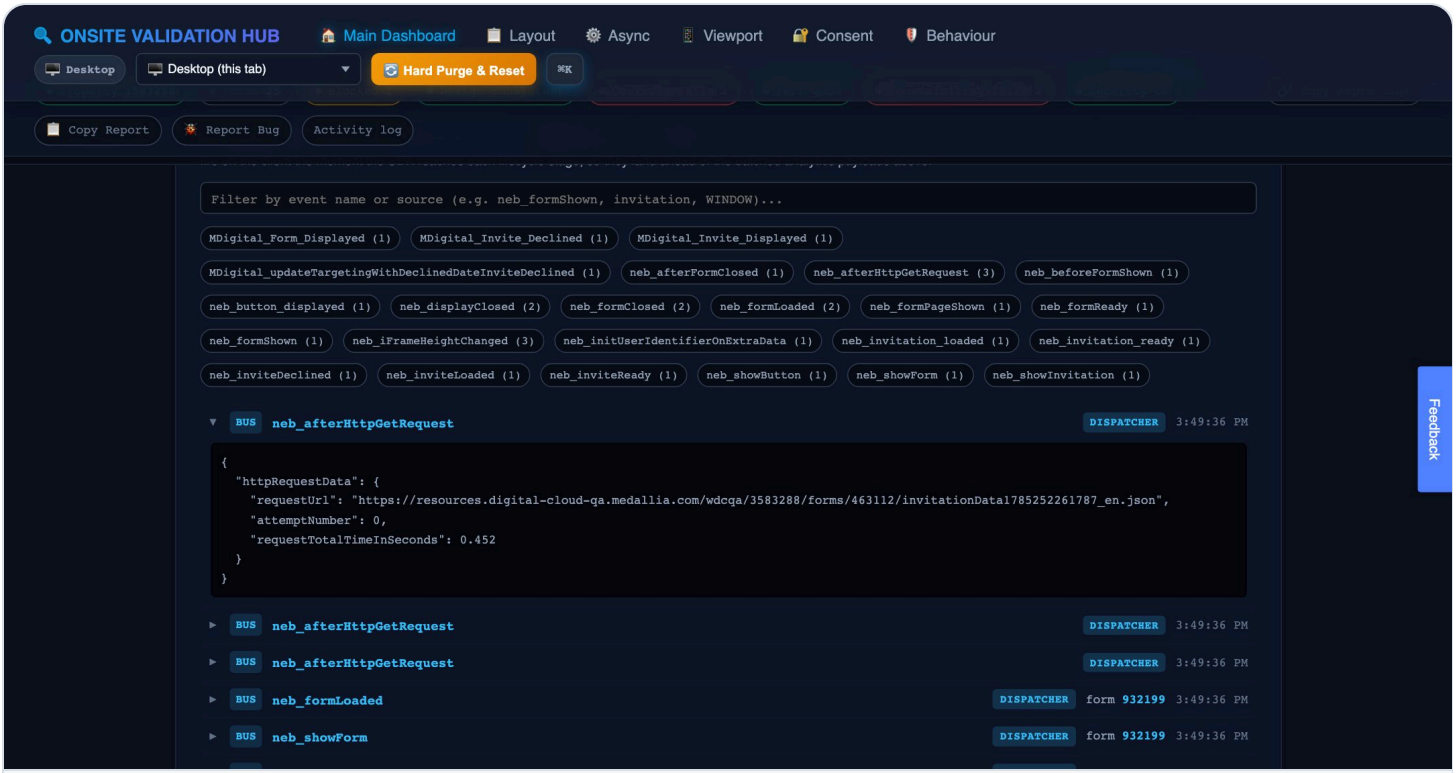
USE THE CHIPS AS A SMOKE TEST

A count climbing into the hundreds while the page just sits there is worth investigating — it usually means something is firing an event in a loop. That is a real defect class, and this panel is the fastest way to notice it.

Native SDK Custom Event Bus

`Panel: Native SDK Custom Event Bus`

A different source from the panel above: these come straight off the SDK's own internal emitter (`KAMPYLE_EVENT_DISPATCHER`) and its `window` CustomEvent rebroadcast — no network involved. They fire the instant the SDK reaches each lifecycle stage, so they arrive *ahead* of the batched analytics payloads and are the better source when you care about ordering and timing.



Lifecycle broadcasts — form loaded, invitation displayed, invite declined — each tagged with its source and form ID.

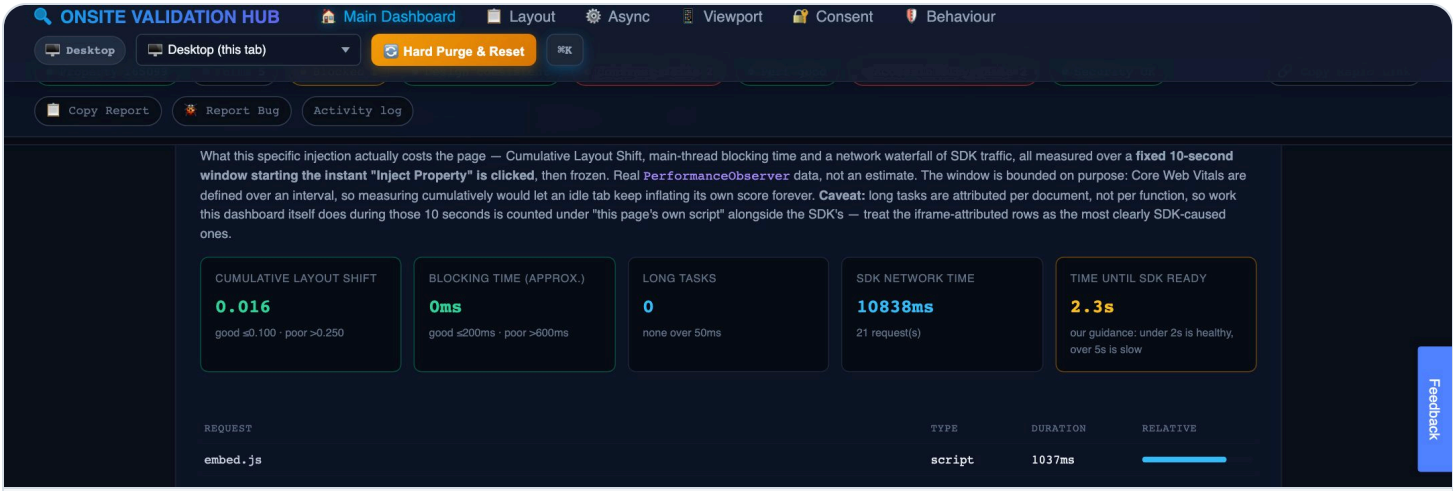
IF THIS PANEL STAYS EMPTY

These broadcasts only exist when the **customEventsBroadcast** provision is enabled for the property. When it's off, nothing you trigger will ever populate this panel — and the panel now says exactly that rather than telling you to re-inject. The **Live Analytics Events** panel above is unaffected either way.

Performance & Core Web Vitals

`Panel: Performance & Core Web Vitals`

CLS, blocking time and long tasks, scoped to exactly the time since *this* property was injected — an idle tab sitting open cannot inflate its own score, and this app's own render loops are excluded from the attribution.

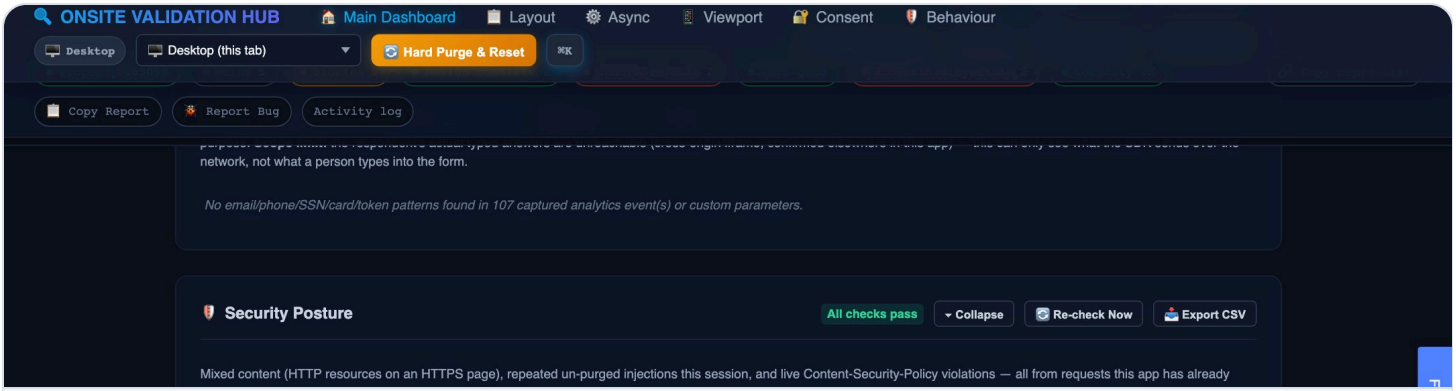


Scoped to the injection window, with a breakdown of what caused each long task.

Privacy / PII Leak Scan

`Panel: Privacy / PII Leak Scan`

Scans every outbound analytics payload and custom parameter for emails, phone numbers, SSNs, card numbers (Luhn-validated, so a random 16-digit session ID doesn't trip a false positive) and API tokens. Matches are shown masked, never as the real value.

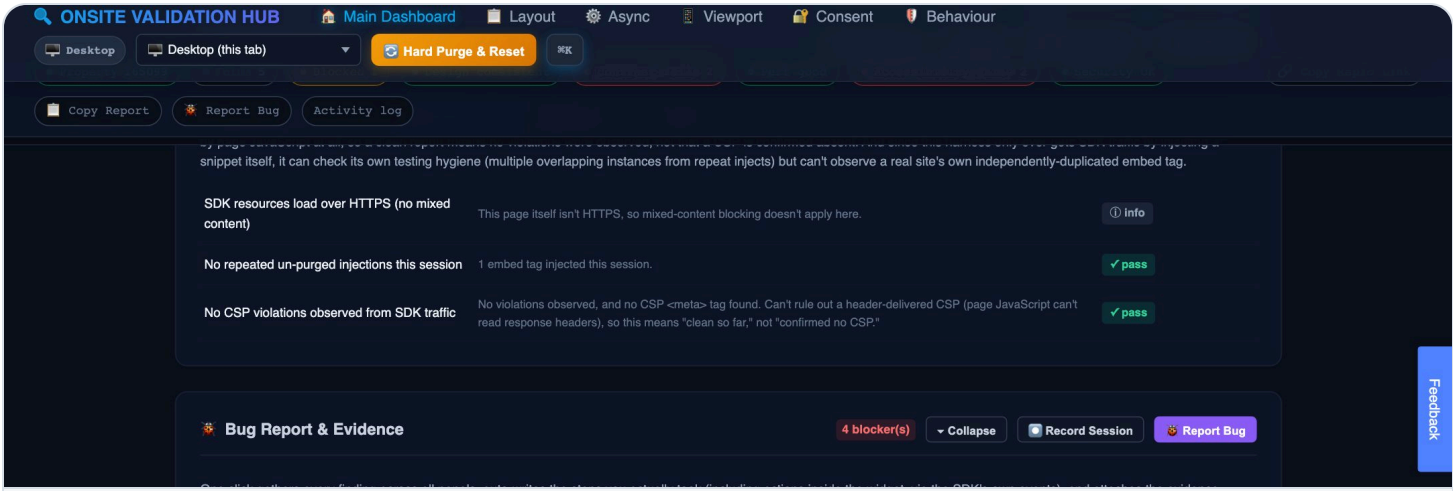


Every captured event scanned automatically — masked results, so the report can't leak what it found.

Security Posture

`Panel: Security Posture`

Mixed content, repeated un-purged injections in the same session, and live CSP violations — all from traffic this app has already confirmed belongs to the SDK. Each check states its own scope limit rather than implying more confidence than it has (for example: a header-delivered CSP isn't readable by page JavaScript at all, so a clean report means "no violations observed," not "no CSP exists").

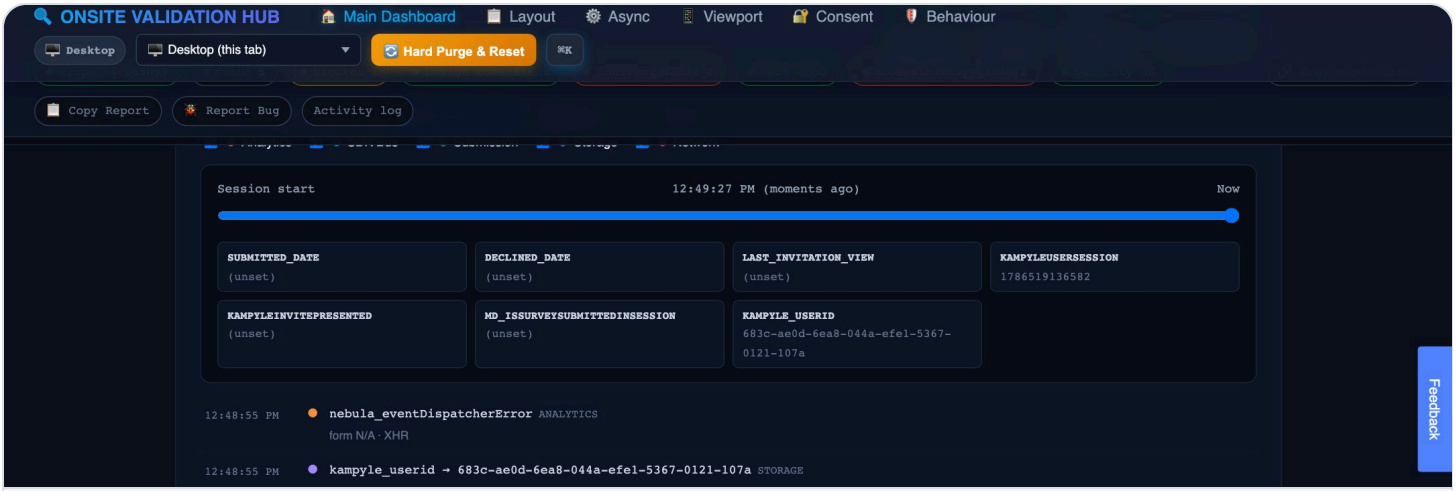


Injection hygiene, mixed content and CSP — checked, with scope limits stated plainly.

Unified Session Timeline

`Panel: Unified Session Timeline`

Every source this app captures — storage writes, analytics events, SDK bus events, submissions — merged into one scrubbable timeline. Filter by source to isolate exactly what happened around a specific moment.

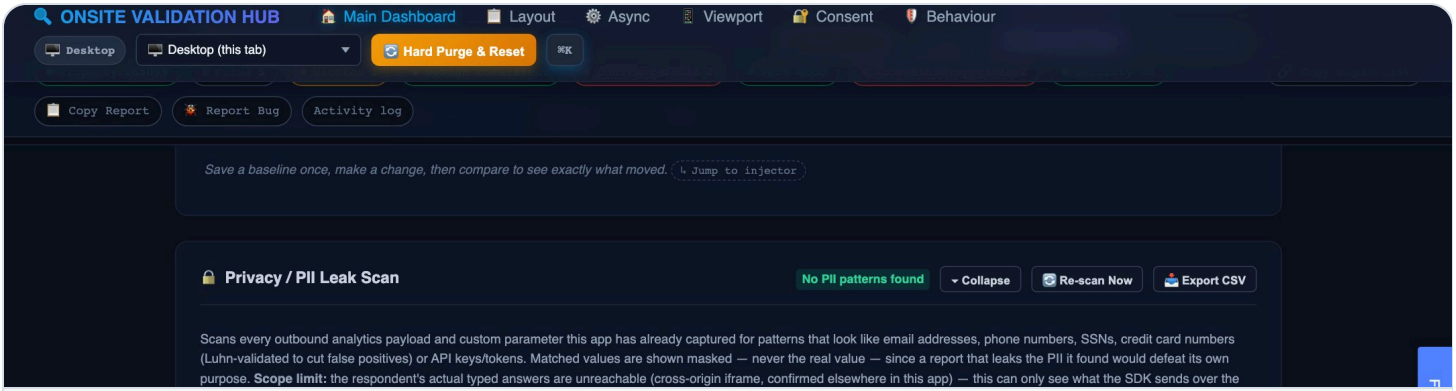


Every captured signal, one timeline, filterable by source.

Findings History

`Panel: What Changed Since Last Run`

Saves a snapshot of current findings as a baseline, then diffs against it on demand — "3 things changed since your last run" is the first thing a returning tester wants, and this is where to get it.

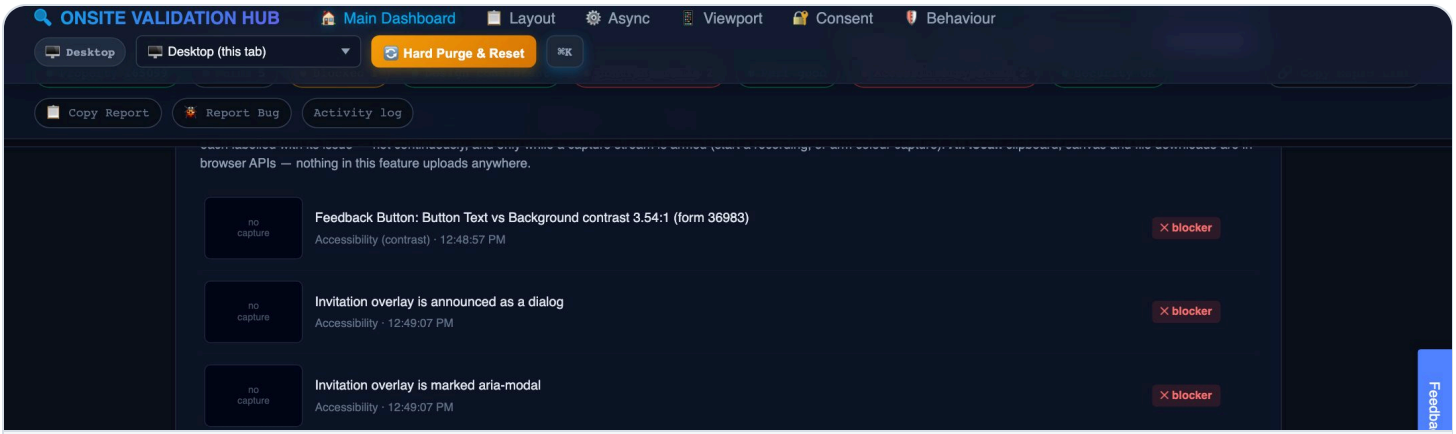


Save a baseline, then see exactly what's new, fixed, or regressed.

Bug Report & Evidence

`Panel: Bug Report & Evidence`

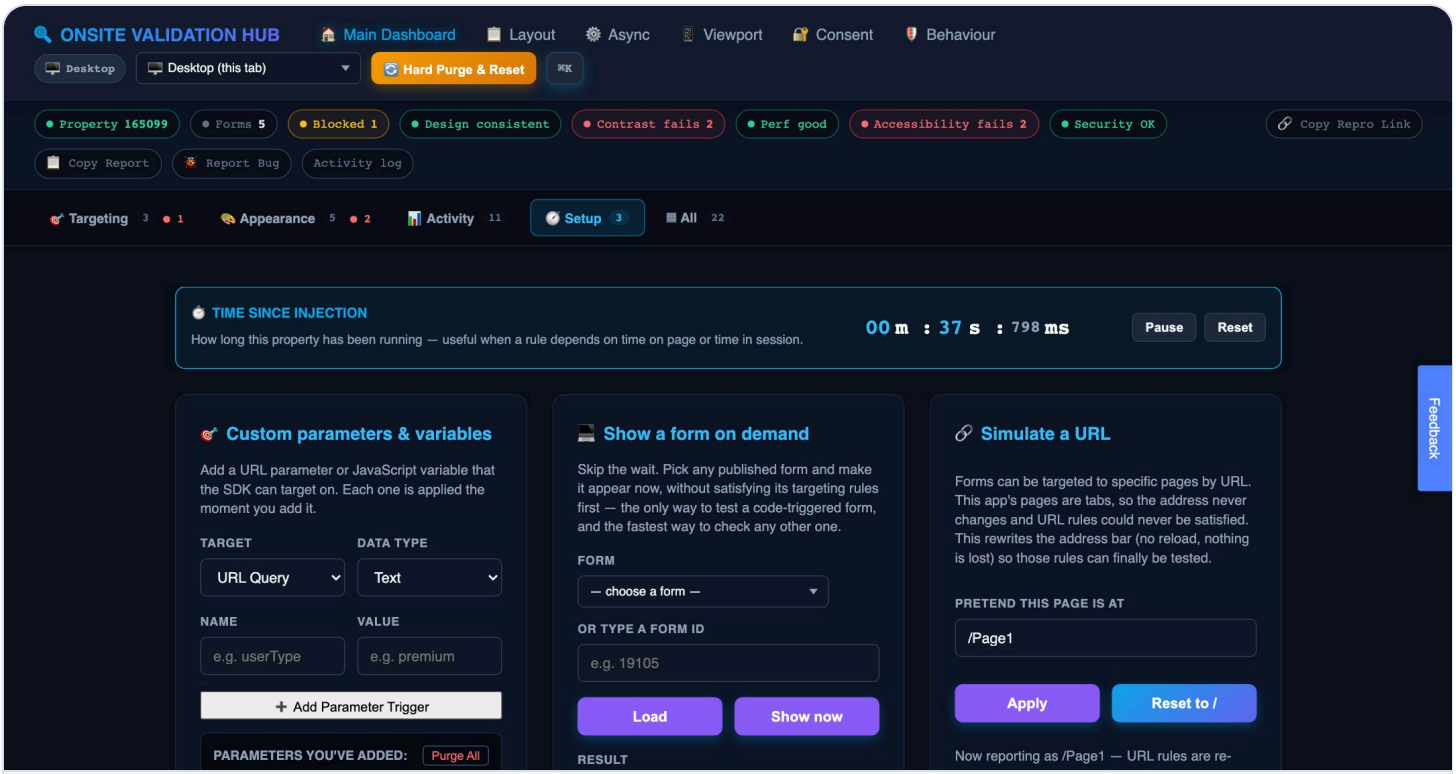
One click gathers every current finding across every panel, writes the steps you actually took (including actions inside the widget, via the SDK's own events), and attaches evidence — screenshots captured automatically the instant a new issue first appeared, not continuously. The result copies to your clipboard and downloads as a self-contained HTML evidence pack. Everything is local: clipboard, canvas and file downloads only — nothing is uploaded anywhere.



Every current blocker, pre-written into a report — ready to paste or download.

WORKSPACE

Setup — configuration and manual control

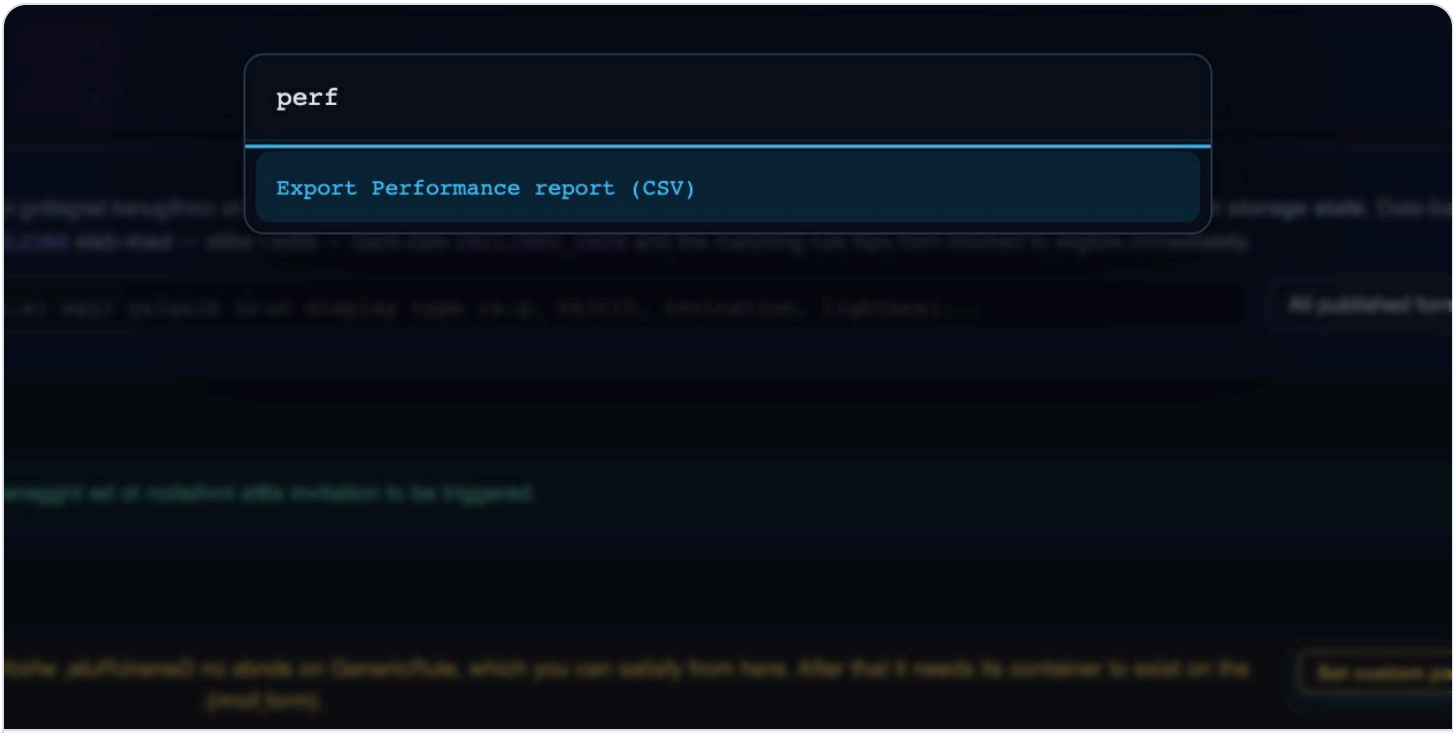


Custom parameters, manual SDK commands, viewport and URL simulation, environment comparisons.

This is where you drive the SDK directly rather than reading its state: set custom parameters and variables the same way a real page would define them, fire `loadForm` / `showForm` / `closeForm` manually to test a code-triggered form on demand, simulate a different URL path without actually navigating, and save named baselines to compare configuration between two environments (e.g. staging vs. production).

Command palette

Press `⌘` `K` (or `Ctrl` `K`) from anywhere to jump straight to a workspace, panel, or export — faster than scrolling and clicking through tabs.



Type to filter; every action in the tool is reachable from here.

ONE THING TO KNOW

Shortcuts go quiet while a form modal has focus — the invitation or form renders inside a cross-origin iframe, so the parent page never receives the keystroke. Click the page background (outside the form) to return keyboard control to the hub. This isn't a bug; it's the same origin boundary that limits what this tool can observe inside the SDK's own frames.

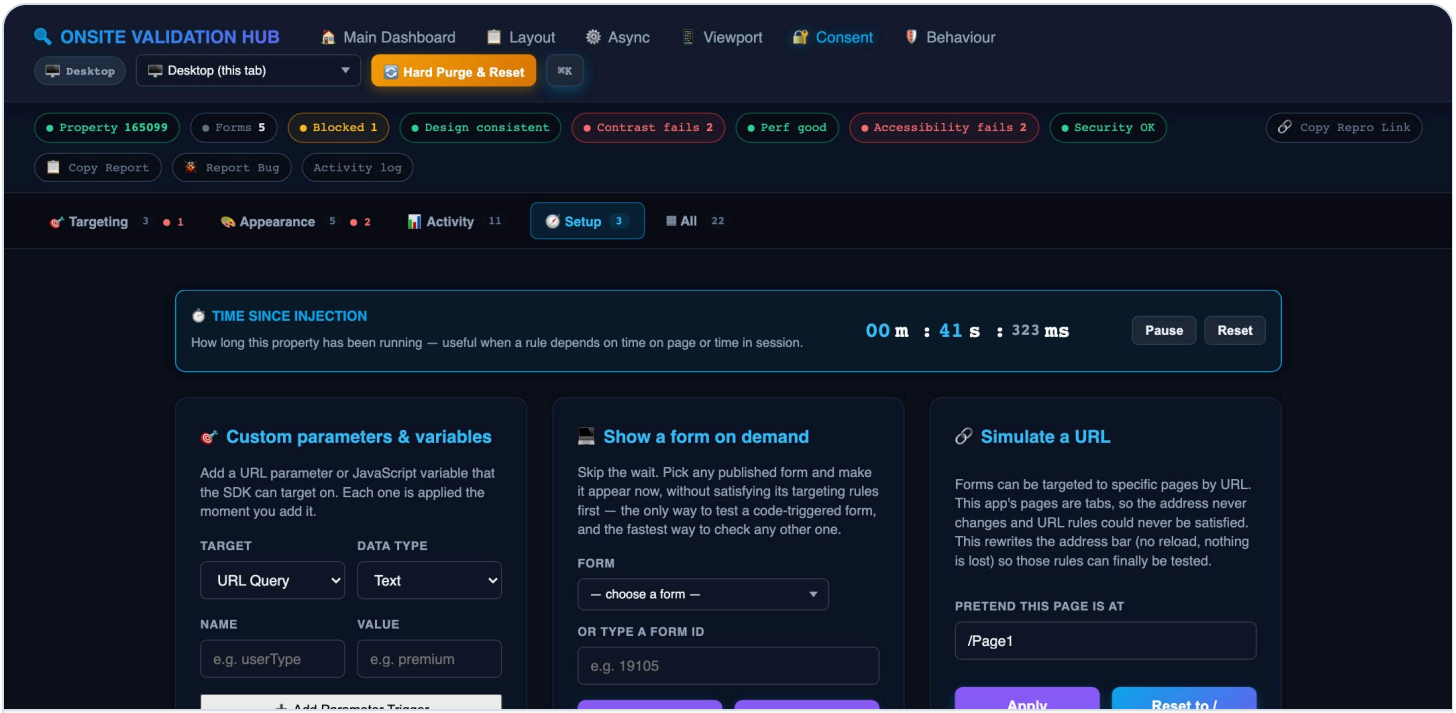
SCENARIO PAGES

Test fixture pages

Five scenario pages simulate the conditions a form actually has to survive in production — an empty sandbox can't reproduce these on its own.

Page 4 — Consent Gating & Data Privacy

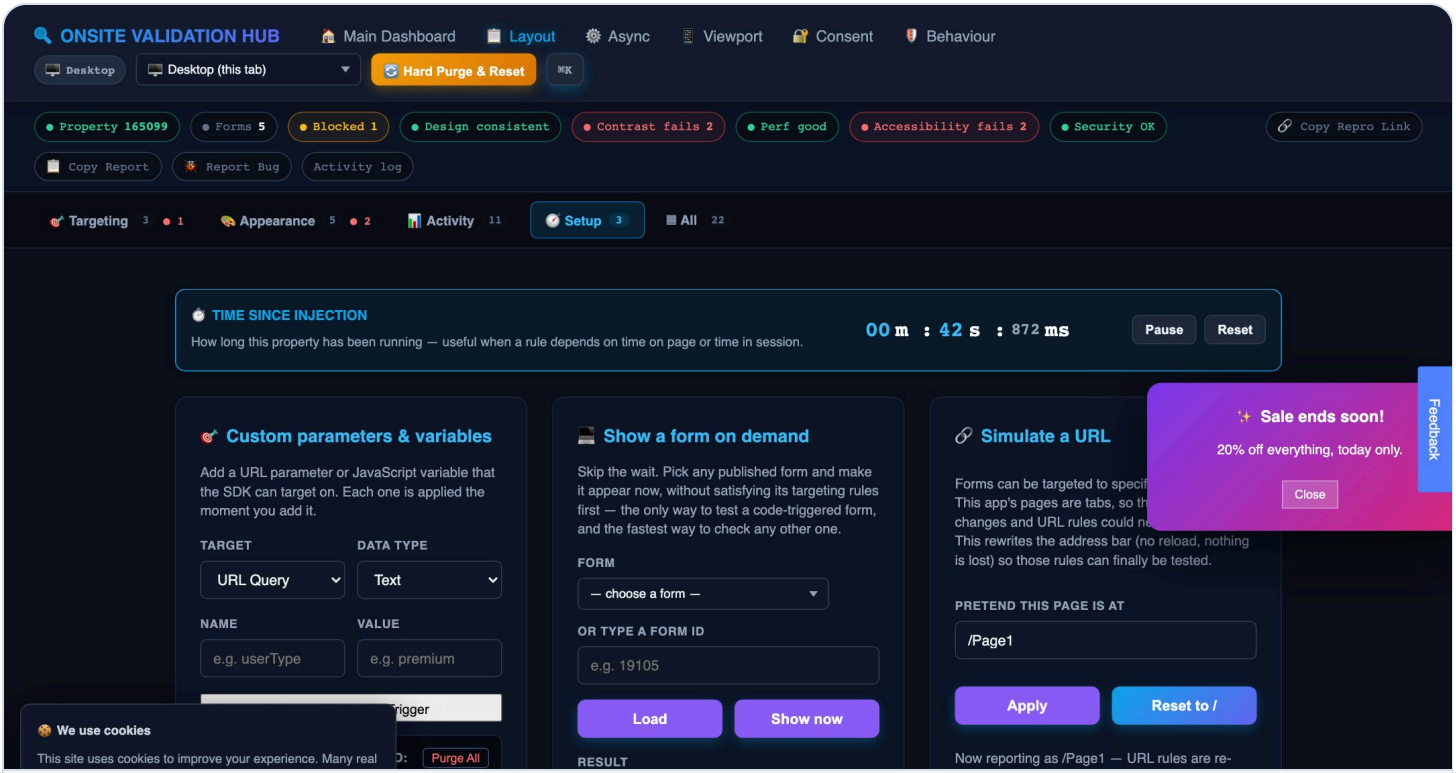
Toggle consent on and off to verify the SDK genuinely respects it — no analytics traffic should fire while consent is denied. Violations are logged explicitly if it doesn't.



Deny consent, then confirm the SDK actually stays silent.

Page 1 — Hostile Layout Fixture

A sticky CTA footer, a cookie banner at maximum z-index, a promo popup decoy, and long scroll content — toggled independently, because a real page fights the onsite widget for space and stacking order, and an empty page can't reproduce that contest.



Every fixture on at once — does the feedback button still work?

The other three scenario pages (**2: Asynchronous Interception**, **3: Adaptive Viewport**, **5: Behavioral Rule Isolation**) are reachable from the Main Dashboard's scenario directory and cover async script timing, zoom/viewport edge cases, and exit-intent style behavioural rules respectively.

RECIPES

Common workflows

Practical answers to the questions people actually ask when they open this tool.

“I just changed a targeting rule — does it actually work?”

1. Inject the property (or Hard Purge & Reset first, if you need a clean visitor).
2. Open the **Targeting** workspace and find the form in the matrix.
3. Read the verdict. If it's blocked, the reason names the exact rule and exact stored value in play.
4. Use the one-click fix if offered — it writes what the rule needs and re-evaluates targeting immediately.
5. Confirm the card shows **LIVE ON PAGE**, not just a green verdict — that's the SDK's own state, not this app's opinion.

“I need to prove a submission actually reached Medallia.”

1. Clear whatever targeting is blocking the form (see the Targeting Matrix), so it genuinely renders.
2. Complete and submit the form exactly as a respondent would.
3. Watch for the **Submitted** chip in the status bar — it appears as soon as the SDK reports it.
4. Click the chip to open the **Feedback Submission Inspector**.
5. Copy the **Feedback UUID** and search it in the Medallia Inbox — it opens that exact response. Paste it into your ticket as proof.

“I need to check accessibility before a release.”

1. Switch to **Appearance**.
2. Open **WCAG Contrast Audit** and **Full Accessibility Audit** — both self-refresh, so trigger the form/invitation you want to check and read the live result.
3. Any check marked ● info couldn't be judged confidently — usually a cross-origin boundary — and is worth a manual look rather than being treated as a pass.

“Something's broken — I need to file a bug fast.”

1. Go to **Activity → Bug Report & Evidence**.
2. Optionally click **Record Session** before reproducing the issue, so a screen capture is attached automatically.
3. Click **Report Bug**. The report and an evidence pack are ready — paste the report text directly into a ticket, and attach the downloaded HTML evidence file.

“I need to test this on an actual mobile device, not just a preview.”

1. Use the device dropdown in the nav bar.
2. **Preview** options check layout only, in this tab.
3. **Launch Chrome** options (need the local `qa-server.py` helper running) open a genuinely emulated mobile Chrome session via DevTools Protocol — this is the only way to get a submission correctly attributed as mobile.

“I want to compare staging against production.”

1. Inject staging, then save it as a named environment in **Setup → Environment A/B Comparison**.
2. Repeat for production.
3. Compare — differences in forms, rules and provisions are listed explicitly.

NON-TECHNICAL READ

For PMs & stakeholders

You don't need to understand localStorage keys or the SDK's internals to get value from this tool — three things are built specifically to be read without technical context.

The status bar

A grey/green row across the top means the property is healthy. Amber or red on any chip — forms blocked, contrast failing, accessibility failing — means something needs a decision, and clicking the chip explains exactly what in plain English.

Copy Report / Session Report

Produces a markdown summary of the whole session: property ID, forms and their verdicts, and every finding — written for someone who wasn't watching the screen, not as a raw data dump.

Copy Repro Link

Packages the current state — including the embed being tested and any simulated URL — into a single link. Send it to a teammate and they land in the exact same state, without re-typing anything.

REFERENCE

Keyboard shortcuts

SHORTCUT	ACTION
`⌘` / `Ctrl` + `K`	Open the command palette
`1` – `5` *(palette open)*	Jump to a workspace — Targeting, Appearance, Activity, Setup, All
`/` *(palette open)*	Filter commands by name
`Esc`	Close the command palette
`Tab`	Every control in the hub, including dynamically generated rows, has an accessible, keyboard-reachable name

---

READ THIS BEFORE YOU DOUBT A RESULT

Known limits

This tool is deliberately honest about what it can't see, rather than guessing and presenting a false verdict. These are structural, not bugs.

CROSS-ORIGIN CONTENT

The invitation and form both render inside a cross-origin iframe. This page can confirm the iframe exists, its size, and whether focus moved into it — but it cannot read what's typed into a field, inspect the form's internal DOM, or verify a handler bound inside that frame (e.g. a real Escape-to-close listener). Checks that hit this boundary say "not a confirmed fail — verify manually" rather than asserting a result they can't back up.

MOBILE PREVIEW VS. MOBILE TARGETING VS. MOBILE SUBMISSION

These are three different scopes. The **preview** checks layout only. The device **dropdown in this tab** can override targeting signals enough to test whether a form *would* show on mobile. Only a **launched, DevTools-driven Chrome session** changes the Client Hints the SDK actually reads for a submission's technical info — spoofing the user-agent string alone (including Chrome's own `--user-agent` flag) still records the real desktop OS and browser version on submit.

BEHAVIOURAL TIMERS AREN'T FIXABLE FROM STORAGE

`TimeOnPage` lives in the SDK's in-memory state, driven by a real timer since page load — there's no storage value to move to fast-forward it. `TimeInSession`, by contrast, is anchored to a stored timestamp and can be tested by adjusting it.

DEVICE TARGETING CAN'T BE SET FROM THIS PAGE

Which device category a form's targeting sees comes from the SDK's own reading of the real user agent at initialisation — nothing this page writes changes it after the fact. Use the device dropdown's tab/launch options, which actually change the browser's reported device, rather than trying to fake it in place.

THE MOBILE CHROME LAUNCHER NEEDS A LOCAL SERVER

`/api/health` and `/api/launch-mobile` only exist when running locally via `qa-server.py` — a browser can't start a process by itself. On a static host (including the deployed URL), this degrades correctly: the option is hidden and replaced with the equivalent terminal command, rather than silently failing.

GETTING TO IT

Where to run it

Run locally with `python3 qa-server.py` — this is required for the mobile Chrome launcher and for deep links (e.g. `/Page1`) to behave exactly as they do in production. A plain static server or the deployed URL both work for everything except the local launcher. See the project `README.md` for the exact commands and current deployment location.

QUESTIONS OR SOMETHING LOOKS WRONG

If a verdict looks wrong, the fastest path is **Activity → Bug Report & Evidence → Report Bug** — it captures exactly the state that produced the confusing result, which is almost always faster to debug from than a description after the fact.

Every screenshot and the walkthrough video above were captured live against a real Medallia property — nothing in this guide is a mock-up or a design comp. If the tool's behaviour ever drifts from what's described here, trust what's on your screen and treat this doc as due for an update.